

Processo de montagem e Ligação e carga



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

O processo de montagem

- Cada máquina (arquitetura) possui uma linguagem de montagem diferente
- O processo de montagem é muito semelhante
- Normalmente utiliza-se os Assemblers de duas etapas

Programa assembly

Rótulo	Opcode	Operandos	Comentários
MARIA:	MOV	EAX, I	$EAX = I$
	MOV	EBX, J	$EBX = J$
ROBERTA:	MOV	ECX, K	$ECX = K$
	IMUL	EAX, EAX	$EAX = I * I$
	IMUL	EBX, EBX	$EBX = J * J$
	IMUL	ECX, ECX	$ECX = K * K$
MARILYN:	ADD	EAX, EBX	$EAX = I * I + J * J$
	ADD	EAX, ECX	$EAX = I * I + J * J + K * K$
STEPHANY:	JMP	DONE	branch to DONE

Assemblers de duas etapas

Rótulo	Opcode	Operandos	Comentários	Comprimento	ILC
MARIA:	MOV	EAX, I	EAX = I	5	100
	MOV	EBX, J	EBX = J	6	105
ROBERTA:	MOV	ECX, K	ECX = K	6	111
	IMUL	EAX, EAX	EAX = I * I	2	117
	IMUL	EBX, EBX	EBX = J * J	3	119
	IMUL	ECX, ECX	ECX = K * K	3	122
MARILYN:	ADD	EAX, EBX	EAX = I * I + J * J	2	125
	ADD	EAX, ECX	EAX = I * I + J * J + K * K	2	127
'STEPHANY:	JMP	DONE	branch to DONE	5	129

O contador de localização de instrução (ILC) monitora o endereço no qual as instruções serão carregadas na memória. Nesse exemplo, as declarações antes de MARIA ocupam 100 bytes.

- Utiliza pelo menos 3 tabelas:
 - Tabela de símbolos
 - Tabela de pseudo-instruções
 - Tabela de opcodes

Tabela de símbolos para o programa da Figura 7.4.

Símbolo	Valor	Outras informações
MARIA	100	
ROBERTA	111	
MARILYN	125	
STEPHANY	129	

- Informações adicionais:
 - Comprimento do campo de dados associado com o símbolo
 - Bits de relocação
 - Símbolo deve ser acessível ou não fora do procedimento

Alguns excertos da tabela de opcodes para um assembler Pentium 4.

Opcode	Primeiro operando	Segundo operando	Opcode hexadecimal	Comprimento da instrução	Classe da instrução
AAA	—	—	37	1	6
ADD	EAX	immed32	05	5	4
ADD	reg	reg	01	2	19
AND	EAX	immed32	25	5	4
AND	reg	reg	21	2	19

Passagem 1 de um assembler simples

```
public static void pass_one() {  
    // Esse procedimento é um esboço de passagem um para um assembler simples  
    boolean more_input = true;           // sinal que pára a passagem um  
    String line, symbol, literal, opcode; // campos da instrução  
    int location_counter, length, value, type; // variáveis diversas  
    final int END_STATEMENT = -2;        // sinaliza final da entrada  
  
    location_counter = 0;                 // monta a primeira instrução em 0  
    initialize_tables();                  // inicialização geral  
  
    while (more_input) {                 // more"input ajustada para falso por END  
        line = read_next_line();         // obtenha uma linha de entrada  
        length = 0;                      // # bytes na instrução  
        type = 0;                        // de que tipo (formato) é a instrução  
    }  
}
```

```

if (line_is_not_comment(line)) {
    symbol = check_for_symbol(line);    // essa linha é rotulada?
    if (symbol != null)                // se for, registre símbolo e valor
        enter_new_symbol(symbol, location_counter);
    literal = check_for_literal(line);  // a linha contém uma literal?
    if (literal != null)                // se contiver, entre a linha na tabela
        enter_new_literal(literal);

    // Agora determine o tipo de opcode – 1 significa opcode ilegal
    opcode = extract_opcode(line);      // localize mnemônico do opcode
    type = search_opcode_table(opcode); // ache formato, por exemplo OP REG1,REG2
    if (type < 0)                       // se não for um opcode, é uma pseudo-instrução?
        type = search_pseudo_table(opcode);
    switch(type) {                      // determine o comprimento dessa instrução
        case 1: length = get_length_of_type1(line); break;
        case 2: length = get_length_of_type2(line); break;
        // outros casos aqui
    }
}

```

```

write_temp_file(type, opcode, length, line); // informação útil para a passagem dois
location_counter = location_counter + length; // atualize loc" ctr
if (type == END_STATEMENT) { // terminamos a entrada?
    more_input = false; // se terminamos, execute tarefas de preparo
    rewind_temp_for_pass_two(); // tais como rebobinar o arquivo temporário
    sort_literal_table(); // e ordenar a tabela de literais
    remove_redundant_literals(); // e remover literais duplicadas
}
}
}

```

Passagem 2 de um assembler simples

```
public static void pass_two( ) {  
    // Esse procedimento é um esboço de passagem dois para um assembler simples.  
    boolean more_input = true;           // sinal que pára a passagem dois  
    String line, opcode;                 // campos da instrução  
    int location_counter, length, type;   // variáveis diversas  
    final int END_STATEMENT = -2;        // sinaliza final da entrada  
    final int MAX_CODE = 16;             // máximo de bytes de código por instrução  
    byte code[ ] = new byte[MAX_CODE];   // contém código gerado por instrução  
  
    location_counter = 0;                 // monta a primeira instrução em 0  
  
    while (more_input) {                  // more_input ajustada para falso por END  
        type = read_type();               // obtém campo de tipo da próxima linha  
        opcode = read_opcode();           // obtém campo de opcode da próxima linha  
        length = read_length();           // obtém comprimento de campo da próxima linha  
        line = read_line();               // obtém a linha de entrada propriamente dita  
    }  
}
```

```

if (type != 0) {                                // tipo 0 é para linhas de comentário
    switch(type) {                              // gerar o código de saída
        case 1: eval_type1(opcode, length, line, code); quebra;
        case 2: eval_type2(opcode, length, line, code); quebra;
        // outros casos aqui
    }
}

write_output(code);                            // escreva o código binário
write_listing(code, line);                     // imprima uma linha na listagem
location_counter = location_counter + length;  // atualize loc_ctr
if (type == END_STATEMENT) {                   // terminamos a entrada?
    more_input = false;                        // se terminamos, execute tarefas de manutenção
    finish_up();                               // avulsas
}
}
}

```

Tabela de símbolos

Codificação hash.

(a) Símbolos, valores e os códigos hash derivados.

(b) Tabela hash de oito entradas com listas encadeadas de símbolos e valores.

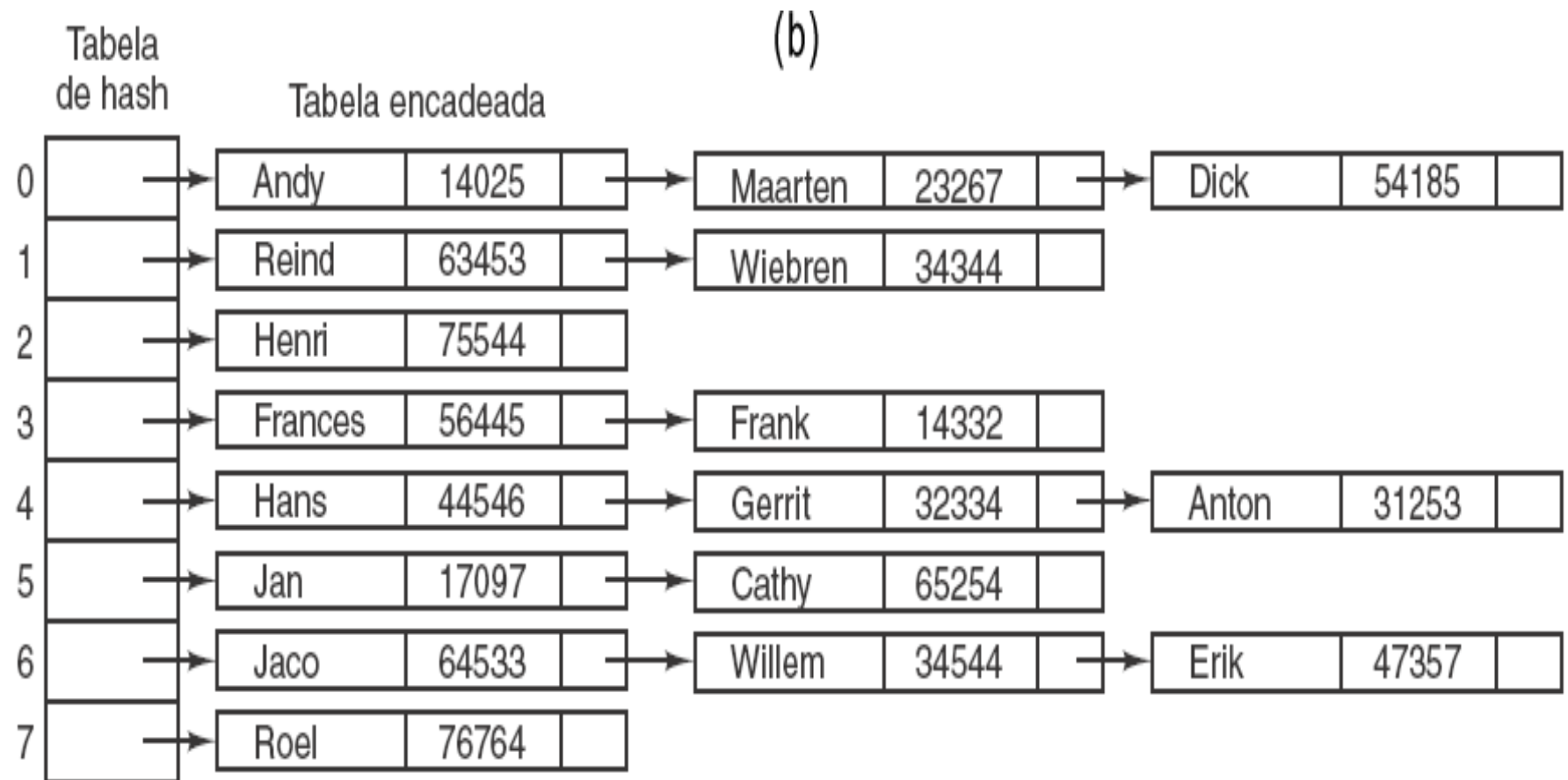
Andy	14025	0
Anton	31253	4
Cathy	65254	5
Dick	54185	0
Erik	47357	6
Frances	56445	3
Frank	14332	3
Gerrit	32334	4
Hans	44546	4
Henri	75544	2
Jan	17097	5
Jaco	64533	6
Maarten	23267	0
Reind	63453	1
Roel	76764	7
Willem	34544	6
Wiebren	34344	1

(a)

Codificação hash.

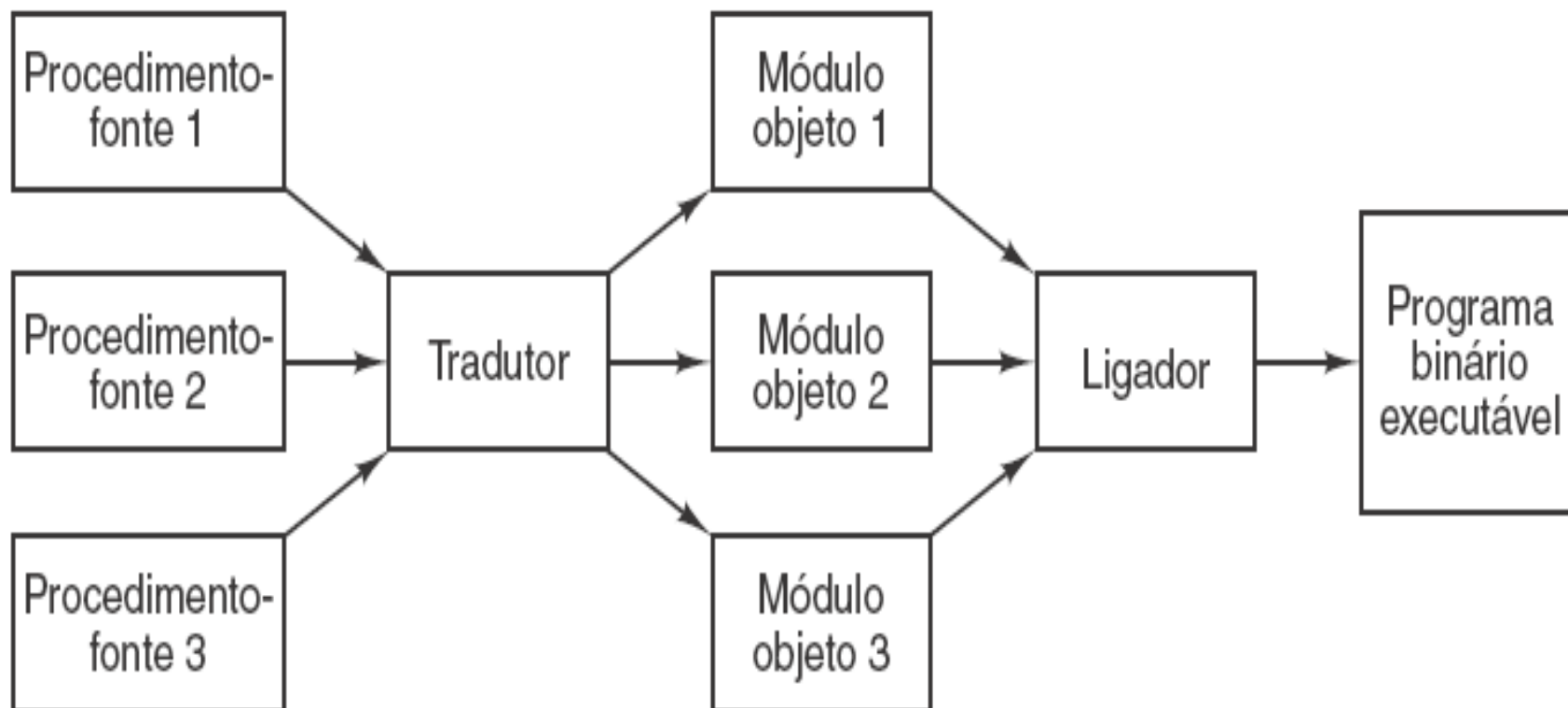
(a) Símbolos, valores e os
códigos hash derivados.

(b) Tabela hash de oito
entradas com listas
encadeadas de símbolos
e valores.



Ligação e carregamento

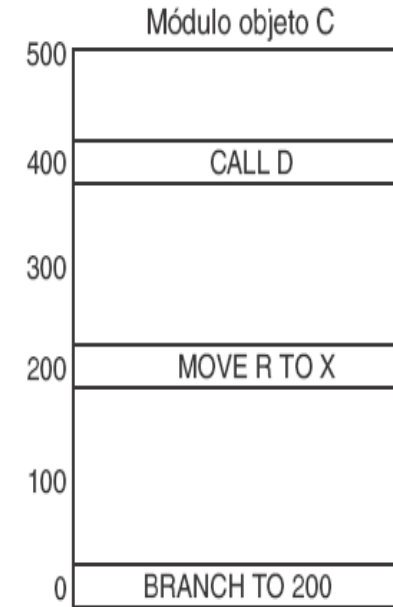
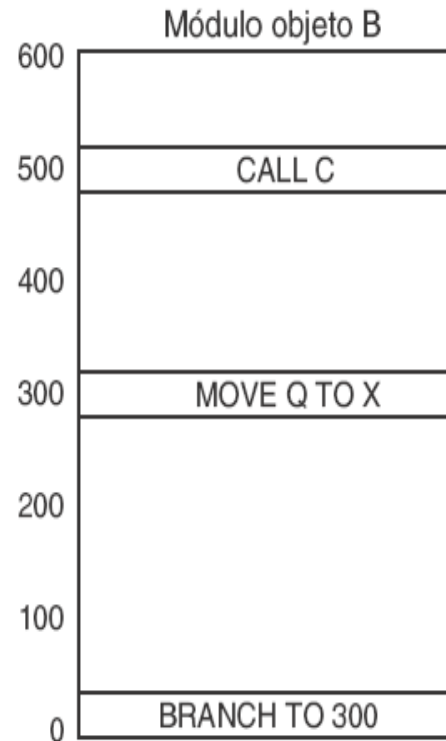
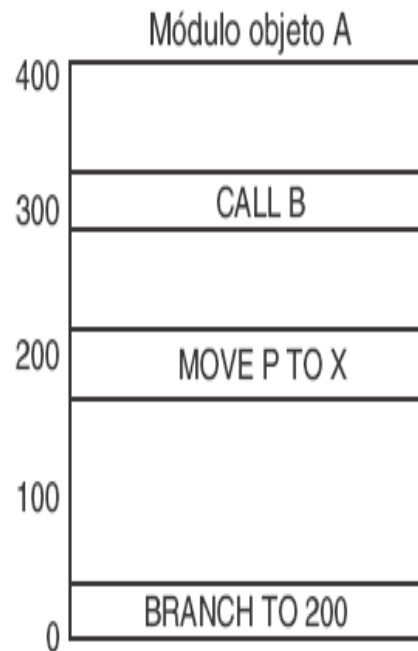
A geração de um programa binário executável a partir de um conjunto de procedimentos-fonte traduzidos independentemente requer a utilização de um ligador.



Tarefas realizadas pelo ligador

Figura 7.9

Cada módulo tem seu próprio espaço de endereço com início em 0.

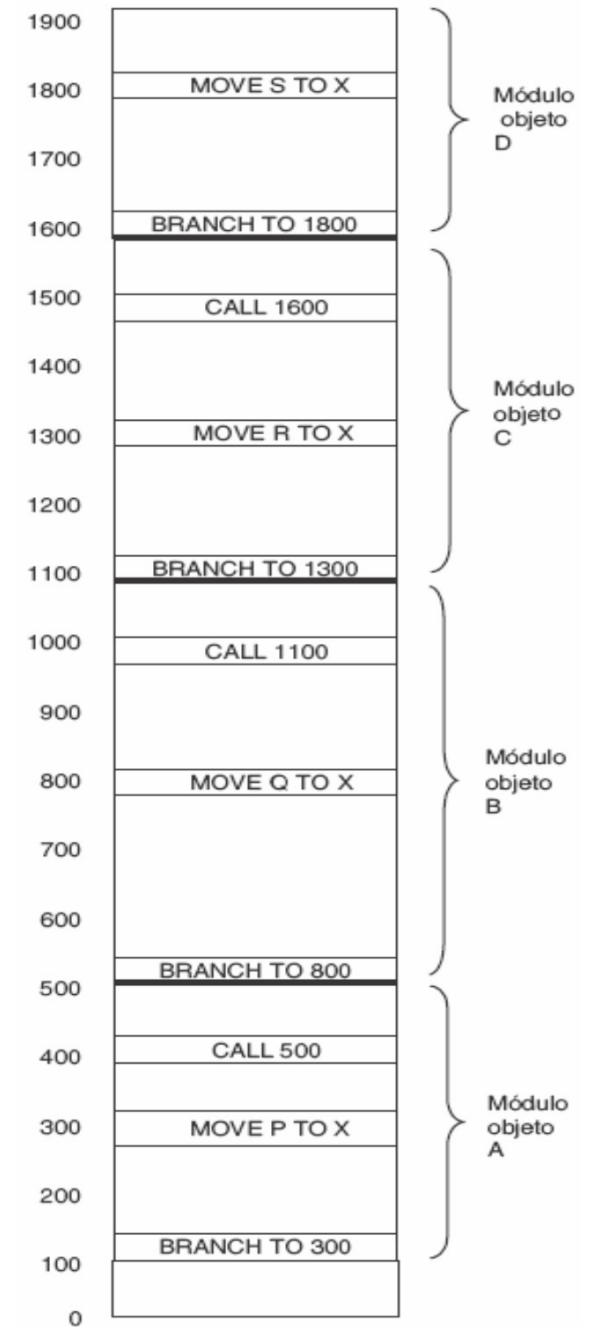
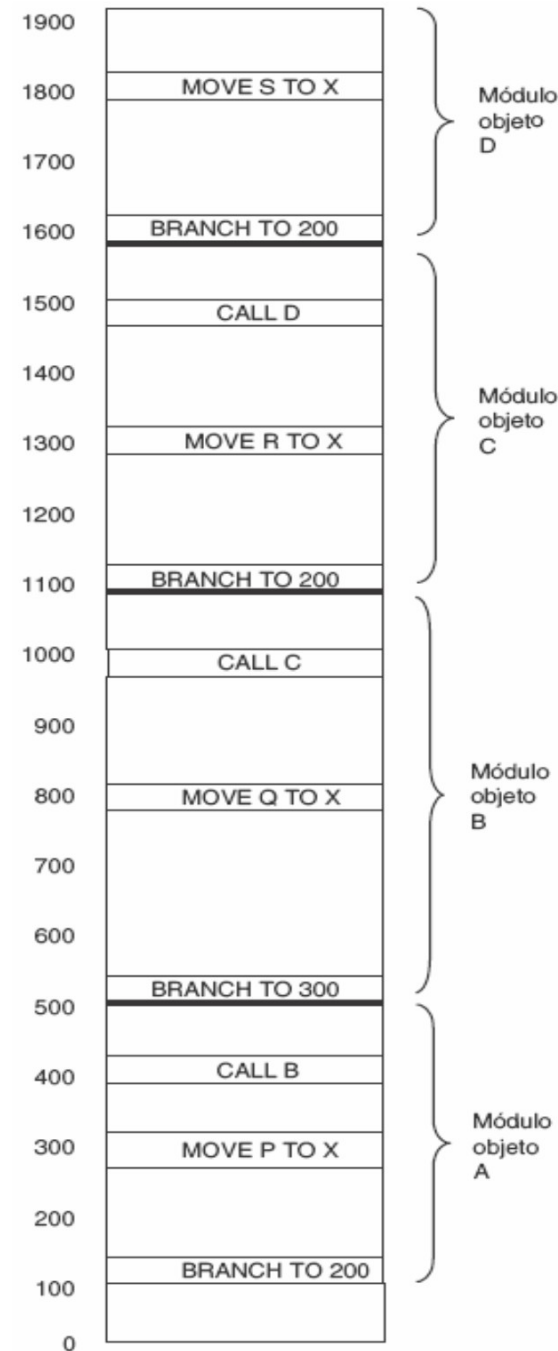


- O ligador funde os espaços de endereços separados dos módulos objeto em um único espaço de endereço linear conforme as quatro etapas seguintes:
 - 1) Ele constrói uma tabela de todos os módulos objeto e seus comprimentos.
 - 2) Com base nessa tabela, o ligador designa um endereço de início de cada módulo objeto.
 - 3) Ele localiza todas as instruções que referenciam memória e adiciona a cada uma delas uma constante de relocação igual ao endereço de início de seu módulo.
 - 4) Localiza todas as instruções que referenciam outros procedimentos e insere os endereços desses procedimentos no lugar adequado.

Tabela construída na etapa 1

Módulo	Comprimento	Endereço de início
A	400	100
B	600	500
C	500	1.100
D	300	1.600

Os módulos objeto da Figura 7.9 após o posicionamento na imagem binária, mas antes da relocação e da ligação.
(b) Os mesmos módulos de objeto após a ligação e após a relocação.

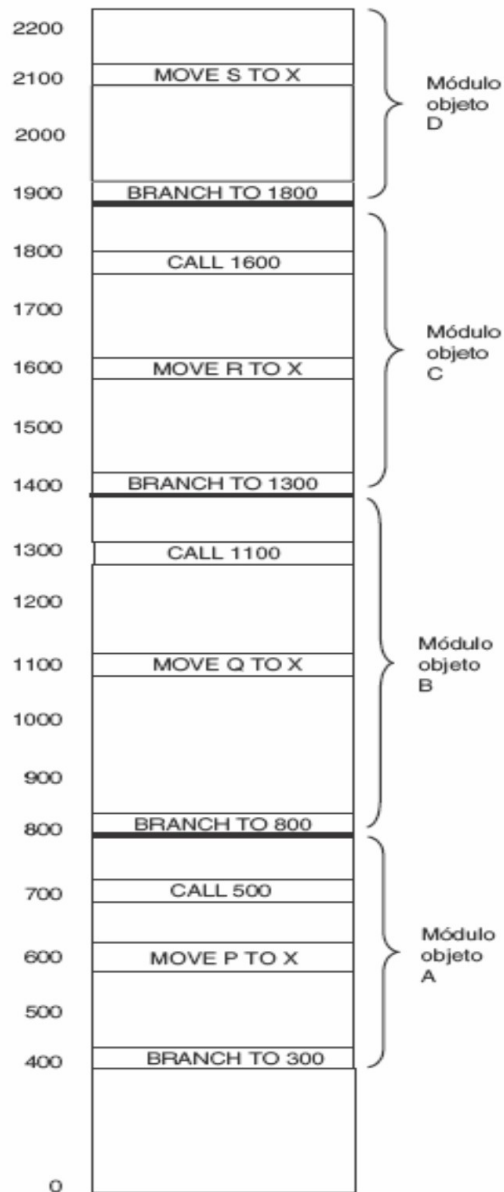


Estrutura de um módulo objeto

Estrutura interna de um módulo objeto produzido por um tradutor.

Final do módulo
Dicionário de relocação
Instruções de máquina e constantes
Tabela de referências externas
Tabela de pontos de entrada
Identificação

Tempo de vinculação e relocação dinâmica



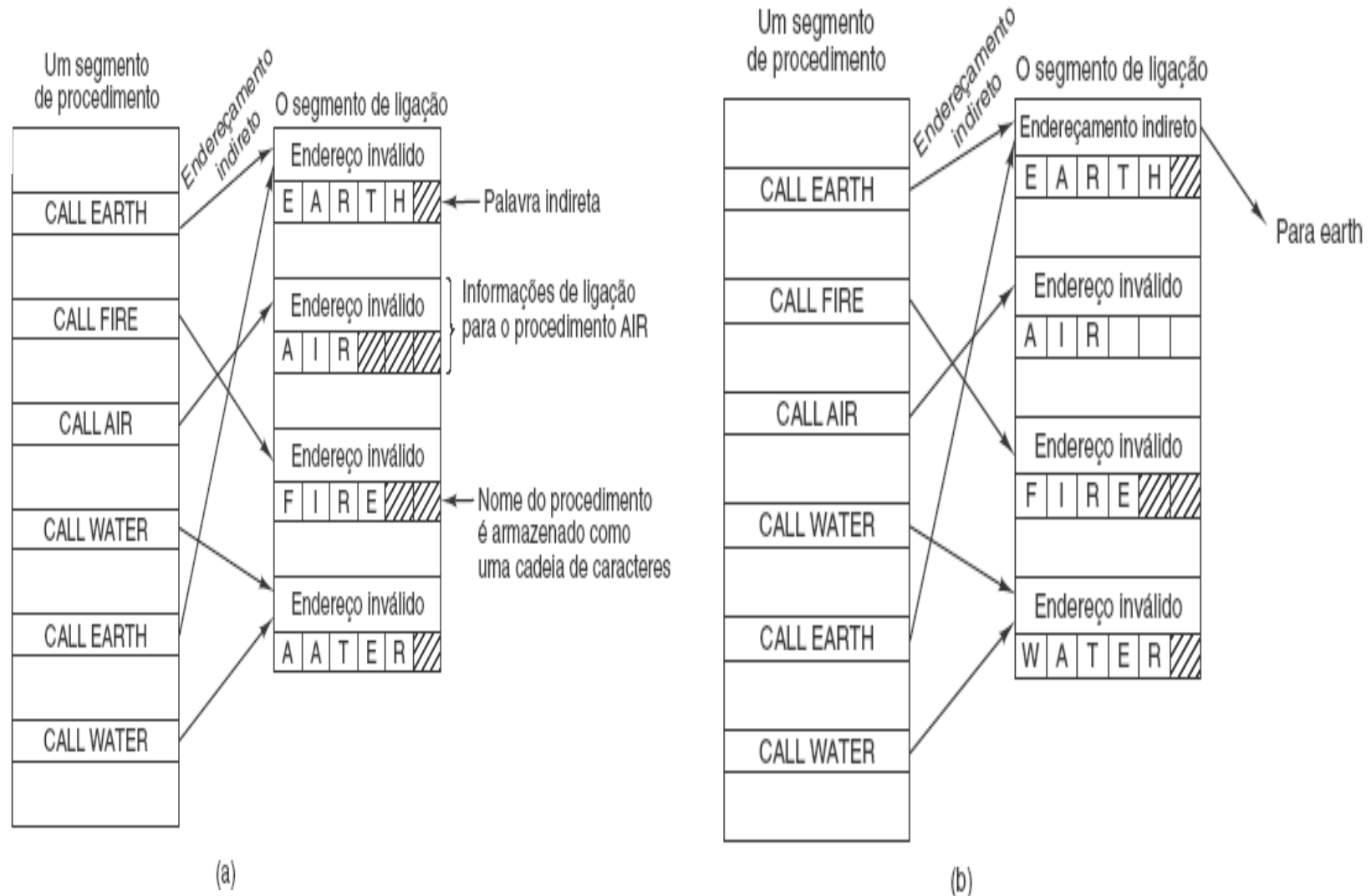
Programa binário relocado da Figura 7.10(b), movido 300 endereços para cima. Muitas instruções agora se referem a um endereço de memória incorreto.

- Há pelo menos seis possibilidades para o momento da vinculação:
 1. Quando o programa é escrito
 2. Quando o programa é traduzido
 3. Quando o programa é ligado, mas antes de ser carregado
 4. Quando o programa é carregado
 5. Quando um registrador de base usado para endereçamento é carregado
 6. Quando a instrução que contém o endereço é executada

Ligação dinâmica no MULTICS

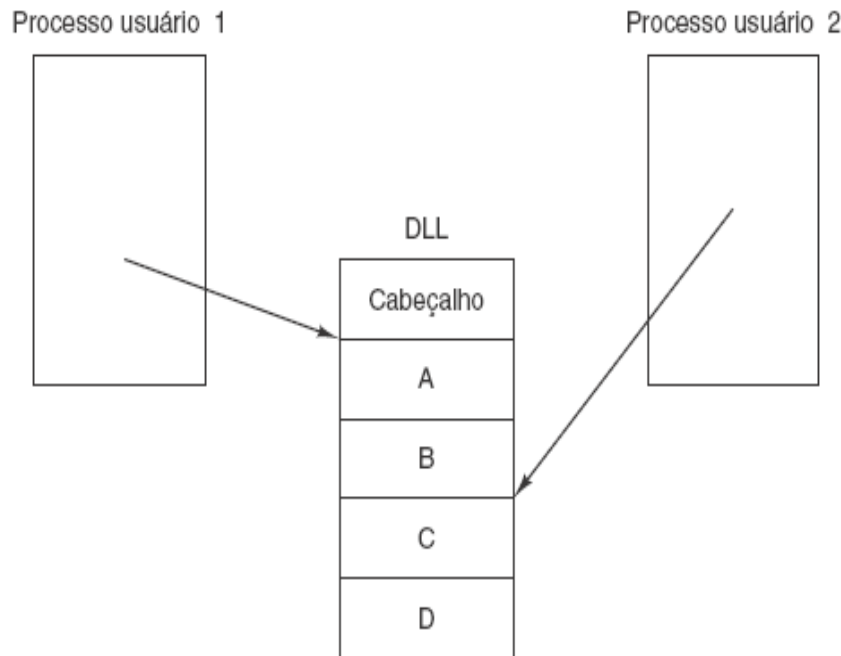
Ligação dinâmica.

(a) Antes de *EARTH* ser chamado. (b) Após *EARTH* ser chamado e ligado.



Ligação dinâmica em Windows

Utilização de um arquivo DLL por dois processos.



- **DLL** – Dynamic Link Library, que podem conter procedimentos, dados ou ambos.
 - DLL's possuem extensão **.dll**
 - Biblioteca de drivers **.drv**
 - Biblioteca de fontes **.fon**

- O **Unix** possui um mecanismo similar às DLLs do Windows, denominado **biblioteca compartilhada** e suporta apenas ligação implícita. Uma biblioteca compartilhada consiste em duas partes: **biblioteca hospedeira**, ligada estaticamente e uma **biblioteca alvo** que é chamada durante a execução.

Bibliografia

- Organização Estruturada de Computadores
– Andrew S. Tanenbaum, Tood Austin – 6ª
Edição 2013 Prentice Hall
- Vários sites da internet